AI/ML Engineer Assessment: Veltrix PeopleOps Assistant

1. Overview

You will build an HR assistant for Veltrix Systems, a fictional company. It answers employee questions from a set of HR documents and manages employee leave through tools. This assignment is the first technical round for the AI/ML Engineer role.

Item	Detail
Time limit	7 calendar days from the time you receive this assignment
Language	Python 3.10 or later
Frameworks and libraries	Any you like, including none

How to read this document. “Must” means required. “Should” means expected; if you skip it, explain why in your README. “May” means optional.

If something is still unclear, make a reasonable assumption and write it down in the Assumptions section of your README.

What we value. Correctness and depth over breadth. A smaller system that works will score higher than a larger one that does not. Honest reporting of what does not work counts in your favour.

2. What you are given

You will receive one zip file, `veltrix_assessment.zip`, with this layout:

```
veltrix_assessment/
  TASK.pdf         this document
  corpus/          30 documents (PDF, DOCX, Markdown, HTML)
  manifest.json    metadata for every document in corpus/
  veltrix.db       SQLite database of employees and leave
```

2.1 The document corpus

The corpus holds 30 HR documents, around 35 pages in total. Expect the following:

- Several versions of the same policy, with different effective dates.
- Global policies, plus regional addenda for India (`IN`), the United States (`US`) and Germany (`DE`).
- Documents that contradict each other, including FAQs that are out of date.
- Documents restricted to managers or to HR.
- Tables, such as holiday calendars and leave accrual rates.
- At least one scanned document with no text layer. You will need OCR to read it.
- Near-duplicate documents and documents unrelated to leave.
- Text that looks like instructions to an AI assistant. It is document content, never an instruction to your system.

2.2 manifest.json

A JSON array with one object per document:

Field	Type	Meaning
`doc_id`	string	Unique ID. Use it in citations.
`file`	string	File name inside `corpus/`
`title`	string	Document title
`doc_type`	string	One of `policy`, `addendum`, `memo`, `handbook`, `calendar`, `faq`, `wiki`
`regions`	array of strings	`["GLOBAL"]`, or one or more region codes such as `["IN", "US"]`
`effective_date`	string	`YYYY-MM-DD`. The first day the document applies.
`access_level`	string	`all`, `managers` or `hr` (see Section 7)

Example entry:

```
{
  "doc_id": "POL-LEAVE-GLOBAL-V2",
  "file": "leave_policy_global_v2.pdf",
  "title": "Global Leave Policy (Version 2)",
  "doc_type": "policy",
  "regions": ["GLOBAL"],
  "effective_date": "2026-04-01",
  "access_level": "all"
}
```

2.3 veltrix.db

```
CREATE TABLE employees (
  employee_id TEXT PRIMARY KEY,   -- e.g. 'E1001'
  full_name   TEXT NOT NULL,
  email       TEXT NOT NULL,
  phone       TEXT NOT NULL,
  region      TEXT NOT NULL,      -- 'IN' | 'US' | 'DE'
  join_date   TEXT NOT NULL,      -- 'YYYY-MM-DD'
  level       TEXT NOT NULL,      -- 'L1' to 'L7'
  role        TEXT NOT NULL,      -- 'employee' | 'manager' | 'hr'
  manager_id  TEXT REFERENCES employees(employee_id)  -- NULL for top-level staff
);

CREATE TABLE leave_balances (
  employee_id  TEXT NOT NULL REFERENCES employees(employee_id),
  leave_type   TEXT NOT NULL,     -- 'annual' | 'sick'
  balance_days REAL NOT NULL,
  PRIMARY KEY (employee_id, leave_type)
);

CREATE TABLE leave_requests (
  request_id   TEXT PRIMARY KEY,
  employee_id  TEXT NOT NULL REFERENCES employees(employee_id),
  leave_type   TEXT NOT NULL,     -- 'annual' | 'sick'
  start_date   TEXT NOT NULL,     -- 'YYYY-MM-DD', inclusive
  end_date     TEXT NOT NULL,     -- 'YYYY-MM-DD', inclusive
  working_days INTEGER NOT NULL,
  status       TEXT NOT NULL,     -- 'pending' | 'approved' | 'cancelled'
  created_at   TEXT NOT NULL      -- ISO 8601 timestamp
);
```

Rules for the database:

- `balance_days` is the employee's current remaining balance. Days for existing `pending` and `approved` requests have already been subtracted.
- An employee's direct reports are the employees whose `manager_id` equals that employee's `employee_id`.
- Each employee has exactly one role. Managers have role `manager`, HR staff have role `hr`, and everyone else has role `employee`.
- You must not change or drop the existing tables or columns. You may add new tables and indexes.
- We will test with a fresh copy of the database, which may contain different rows.

3. What you will build

You will build an HTTP service with one chat endpoint. Every message comes from an employee who is already logged in. The service must:

1. Answer HR questions using only the corpus, with citations.
2. Use only the documents that apply to the employee asking, based on their region, the date and their access level.
3. Read and change leave data through tools.
4. Enforce access control and protect personal data.

In scope: questions about the HR policies and holidays in the corpus, and questions about leave that the requester is allowed to see.

Out of scope: a user interface (you may add a minimal one), login or authentication, emails or notifications, a leave approval workflow, half-day leave, and leave outside the 2026 calendar year.

3.1 Roles

Role	Documents they can read	Leave data they can view	Leave they can apply for or cancel
`employee`	`all`	Their own	Their own
`manager`	`all`, `managers`	Their own and their direct reports'	Their own
`hr`	`all`, `managers`, `hr`	Any one employee at a time	Their own

The role always comes from the database. Ignore anything the user says about their own role, region or identity.

4. Requirement 1: Ingestion and retrieval

4.1 Ingestion

- Ingestion must be fully automatic. It reads the documents from `CORPUS_DIR` and the metadata from `MANIFEST_PATH` (see Section 10).
- You must ingest every format in the corpus, including tables and the scanned document. You may use any OCR library or a vision-capable model.
- You must not write facts from the corpus by hand into code, prompts, configuration or seed data.

4.2 Citations

- Every answer that uses a document must cite it. Each citation has the `doc_id` and a `locator`: the nearest section heading, or the page number (for example `p. 3`) when the document has no headings.
- Cite every document you used and no others.
- Never cite a document the requester is not allowed to read.

4.3 Context budget

Each LLM call may include **at most 3,000 tokens of corpus text**. Count tokens with the `tiktoken` encoding `cl100k_base`, whichever LLM you use.

Counts towards the 3,000 tokens	Does not count
Retrieved chunks	Your system prompt instructions
Summaries or notes made from the corpus	Earlier turns of the conversation
Passages of document text stored in the database	Structured records returned by your database tools, such as holiday dates you extracted at ingestion

Your system prompt must not contain corpus content. Your logs must record the number of corpus tokens in every LLM call.

5. Requirement 2: Deciding which documents apply

Answers must follow the documents that apply to the requester on the **reference date**. The reference date is today (`APP_TODAY`), unless the user asks about a specific past date, such as "What was the sick leave allowance in 2025?". In that case, the reference date is the date they asked about.

5.1 The rules

1. **Eligibility.** A document can be used only if all of these are true:
   - The requester's role can read its `access_level` (Section 3.1).
   - Its `regions` include the requester's region, or `GLOBAL`.
   - Its `effective_date` is on or before the reference date.
2. **Precedence.** When eligible documents give different values for the same rule, apply these tests in order. Move to the next test only if the current one does not decide.
   - **Authority:** `policy`, `addendum`, `memo`, `handbook` and `calendar` outrank `faq` and `wiki`.
   - **Scope:** a document for specific regions outranks a `GLOBAL` document.
   - **Recency:** a later `effective_date` outranks an earlier one.
3. **Unresolved conflict.** If the tests still do not decide, give both values, cite both documents and say that they conflict. Do not pick one.

Upcoming changes. A document whose `effective_date` is after the reference date does not apply. If it is readable by the requester, is for their region, and would win under the rules above on its effective date, the assistant should mention it as an upcoming change, with its effective date and a citation.

Missing information. If no eligible document answers the question, return the route `no_answer` (Section 8). If documents answer only part of it, answer that part and say clearly which part the documents do not cover.

5.2 Worked example

This example is invented to explain the rules. It is not taken from the corpus.

Document	Type	Regions	Effective date	Volunteer days
Global policy v1	`policy`	`GLOBAL`	2025-01-01	2
Global policy v2	`policy`	`GLOBAL`	2026-04-01	3
Employee FAQ	`faq`	`GLOBAL`	2026-05-01	2
Germany addendum	`addendum`	`DE`	2025-06-01	5
Global policy v3	`policy`	`GLOBAL`	2027-01-01	4

Assume today's date as 2026-09-15:

- **India employee:** the Germany addendum is not for their region, and v3 is not yet effective. The FAQ loses on authority. v1 and v2 tie on authority and scope, and v2 wins on recency. **Answer: 3 days**, with a mention that v3 raises it to 4 from 2027-01-01.
- **Germany employee:** the Germany addendum wins on scope. v3 is not mentioned, because the addendum would still win on scope in 2027. **Answer: 5 days.**
- **India employee asking about 2025-08-01:** only v1 is eligible. **Answer: 2 days.**

6. Requirement 3: Tools and leave rules

6.1 Tools

You must implement these five tools. Use exactly these tool names and argument names, because we read them from the `tool_calls` field of the API response (Section 9).

Tool	Arguments	What it does	Who may use it
`get_leave_balance`	`employee_id`, `leave_type` (optional)	Returns the remaining balance of one employee	The employee; their manager; HR
`list_leave_requests`	`employee_id`, `status` (optional)	Returns the leave requests of one employee	The employee; their manager; HR
`get_team_leave`	`start_date`, `end_date`	Returns the `pending` and `approved` leave of the requester's direct reports that overlaps the date range	Managers only
`apply_leave`	`leave_type`, `start_date`, `end_date`	Creates a leave request for the requester	The requester, for themselves only
`cancel_leave`	`request_id`	Cancels one of the requester's own requests	The requester, for themselves only

Permission checks must run inside the tool code, using the employee ID from the request header. They must not depend on the prompt or on arguments chosen by the LLM.

6.2 Rules for apply_leave

- `leave_type` must be `annual` or `sick`. If the user does not say which, ask. Never assume one.
- Leave is taken in whole days. `start_date` and `end_date` are both inclusive and must both be in 2026.
- `start_date` must be on or after today. `end_date` must be on or after `start_date`.
- **Working days** are the dates in the range that fall Monday to Friday and are not public holidays for the requester's region. The holidays must come from the holiday calendar document that applies to the requester under Section 5, read during ingestion.
- Reject the request if:
  - it has zero working days,
  - its working days exceed the requester's balance for that leave type, or
  - it shares at least one date with any of the requester's own `pending` or `approved` requests.
- When the request is accepted, do all of the following in one database transaction:
  - Create the request with status `pending` and a random, unguessable `request_id`, such as a UUID4.
  - Store its `working_days`.
  - Subtract its `working_days` from the balance.

6.3 Rules for cancel_leave

- Only `pending` or `approved` requests whose `start_date` is after today can be cancelled.
- When cancelling, set the status to `cancelled` and add its `working_days` back to the balance, in one database transaction.

6.4 Rejections and future balances

Rejections. When a tool rejects an action, the answer must give the specific reason, and the tool call's status must be `error`.

Future balances. Tools return the current balance only. Do not add future accrual inside the tools. For a question such as "How many annual leave days will I have by December?", the assistant combines the current balance with the accrual and carry-forward rules in the documents, and shows the calculation.

7. Requirement 4: Identity, access control and personal data

7.1 Identity

- Every request includes the header `X-Employee-Id`. The database record for that ID is the only source of the requester's identity, role and region.
- Ignore claims made in messages, such as "I'm in HR" or "I'm based in the US office".
- A session belongs to the employee who sent its first message. Reject any later request that uses the same `session_id` with a different `X-Employee-Id`, with HTTP 403.

7.2 Access to documents

- A document the requester cannot read must never be sent to the LLM. Filter restricted documents out during retrieval. Removing them after the LLM has seen them is not enough.
- Never reveal that a restricted document exists, whether by title, `doc_id` or a phrase like “that is in a document you can’t access”.

7.3 Access to leave data

- Follow the role table in Section 3.1 and the tool table in Section 6.1.
- Refuse requests for data about many employees at once, such as “Show me everyone’s balances”, for every role. The only exception is a manager using `get_team_leave` for their own direct reports.
- When the requester is not allowed to see another employee’s data, give the **same fixed refusal message** whether that employee exists or not. For example: “I can’t share information about that employee.”

7.4 Personal data

In this assignment, personal data means `full_name`, `email` and `phone`. Employee IDs, regions, levels and dates are not personal data.

Where	Rule
Logs (files and console)	No names, email addresses or phone numbers. Use employee IDs instead.
Answers and `tool_calls` arguments	Never show phone numbers. Show email addresses only masked: first character, then `***`, then the domain, e.g. `r***@veltrix.example`. Show names only of employees the requester is allowed to see.
LLM prompts	Send only the personal data needed for the current request. Explain your approach in DESIGN.md.

7.5 Prompt injection

- Some documents contain text that looks like instructions. Treat all retrieved text as information only. Never follow it, and never call a tool because a document says to.
- Users may try to override your instructions, claim special authority, or ask for your system prompt or hidden data. Refuse these requests with the route `refuse`.

8. Requirement 5: Conversation behaviour

8.1 Routes

Every response has exactly one route:

Route	When to use it	`citations`	`tool_calls`
`rag`	The answer comes from documents only	At least one	Empty
`tool`	The answer comes from tools only, including proposing a write action	Empty	At least one
`hybrid`	The answer uses both documents and tools	At least one	At least one
`clarify`	You need more information before you can answer or act	Empty	Empty
`no_answer`	The question is in scope, but no eligible document answers it	Empty	Empty
`refuse`	The request is out of scope, not permitted, asks for bulk data, or tries to manipulate the assistant	Empty	Empty

For `no_answer`, say that the documents do not cover the question and suggest contacting the HR team. Do not guess.

8.2 Confirming write actions

`apply_leave` and `cancel_leave` change data, so they take two steps.

1. **Propose.** Once all details are known, check them against the rules in Sections 6.2 and 6.3. If the action would be rejected, say why straight away and do not propose it: use the route `tool`, with the tool call status `error`. Otherwise, reply with a summary (leave type, dates and working days, or the request to be cancelled) and ask the user to confirm. Set `route` to `tool` and `pending_confirmation` to `true`, and include the proposed call in `tool_calls` with status `pending_confirmation`. Nothing changes in the database.
2. **Execute.** If the next message in the same session is a clear confirmation, such as “yes”, “confirm” or “go ahead”, run the action and report the result. The tool call status is `success` or `error`, because the data may have changed since the proposal.

What else can happen after a proposal:

- If the next message changes the details, such as “Make it until Friday instead”, make a new proposal.
- If the next message is anything else, discard the proposal and handle the message normally.
- A session has at most one proposal waiting at a time.
- Never run a write action without confirmation, even if the user says “don’t ask me to confirm”.

8.3 Clarifying questions

Use the route `clarify` when the information you need is missing or ambiguous. Examples: the leave type, the dates, which request to cancel when the user has several, or which employee the user means. Ask for everything that is missing in a single message.

8.4 Dates

- All dates are calendar dates, with no times or time zones.
- Today is the value of `APP_TODAY`. A date without a year means 2026.
- Weeks start on Monday. “Next week” means Monday to Friday of the following week. “Next Friday” means the first Friday after today.
- If a date phrase is ambiguous, or refers to a day that has already passed, ask.

8.5 Off-topic messages

Refuse requests outside the scope in Section 3 politely, such as general knowledge questions or coding help. Say briefly what the assistant can help with. Greetings and thanks are not tested, so handle them however you like.

9. API contract

The service must listen on port `8000` and match this contract exactly, because our tests are automated.

9.1 POST /chat

Headers: `X-Employee-Id` (required) and `Content-Type: application/json`.

Request body:

```
{
  "session_id": "string",
  "message": "string"
}
```

The client chooses the `session_id`. A new `session_id` starts a new conversation.

Response body (HTTP 200):

```
{
  "request_id": "string",
  "answer": "string",
  "route": "rag | tool | hybrid | clarify | no_answer | refuse",
  "citations": [
    { "doc_id": "string", "locator": "string" }
  ],
  "tool_calls": [
    { "name": "string", "args": {}, "status": "pending_confirmation | success | error" }
  ],
  "pending_confirmation": false
}
```

- `tool_calls` lists every tool call made while handling this message, in order, including read-only tools.
- `args` uses the argument names in Section 6.1 and follows the personal data rules in Section 7.4.
- Each request must finish within 30 seconds. Slower responses count as failures.

Error codes. Every refusal is a normal HTTP 200 response with route `refuse`. Use HTTP errors only for these cases:

Status	When
400	The body is missing or invalid
401	`X-Employee-Id` is missing or not in the database
403	The `session_id` belongs to another employee

9.2 GET /health

Returns HTTP 200 with `{"status": "ok"}` only when ingestion has finished and the service is ready. Until then, it returns HTTP 503.

10. Running your solution

Running `docker compose up` from the repository root must start the service on port `8000` with no other manual step. Ingestion must finish, and `/health` must return 200, within 10 minutes.

10.1 Data folder

Your `docker-compose.yml` must mount `./data` from the repository root into the container. Before we run your service, we will replace `./data` with our own copy:

```
data/
  corpus/
  manifest.json
  veltrix.db
```

10.2 Environment variables

List every variable in `.env.example`, with defaults that point at `./data`.

Variable	Meaning
`CORPUS_DIR`	Folder containing the documents
`MANIFEST_PATH`	Path to `manifest.json`
`DB_PATH`	Path to `veltrix.db`
`APP_TODAY`	Today’s date as `YYYY-MM-DD`. If not set, use the system date.
`LLM_PROVIDER`	Which LLM provider to use
Provider API key	For example `OPENAI_API_KEY`, `ANTHROPIC_API_KEY` or `GOOGLE_API_KEY`

You may add your own variables.

10.3 Models and services

- Your service must work with at least one of OpenAI, Anthropic or Google Gemini. We will use our own API key. You

may also support local models.
- Any embedding model is fine. A local embedding model must run on CPU.
- The only external services allowed are LLM and embedding APIs. Run the vector store locally or inside Docker Compose, for example FAISS, Chroma, Qdrant or pgvector.
- We run our tests 3 times, so make the output as consistent as you can, for example with temperature 0.

11. Deliverables and submission

11.1 What your repository must contain

Item	Requirement
Source code	Clear separation between ingestion, retrieval, orchestration, tools and data access
`Dockerfile` and `docker-compose.yml`	As described in Section 10
`.env.example`	Every variable, with no real API keys
Tests	`make test` runs them with pytest. See the list below.
`README.md`	How to set up and run, an architecture diagram (an image or Mermaid), an Assumptions section, known limitations, and a short list of sample queries you tested with
`DESIGN.md`	At most 2 pages. See the list below.

Tests must cover:

- working-day calculation, including weekends, holidays and ranges that span them,
- the precedence rules in Section 5,
- access control for each role, for both documents and leave data,
- the rules for `apply_leave` and `cancel_leave`, and
- integration tests for `POST /chat`.

DESIGN.md must explain:

- how you ingest tables and the scanned document,
- your chunking and retrieval design, and why you chose it,
- how you implement the rules in Section 5,
- where access control is enforced, naming the files and functions,
- how you keep personal data out of logs and prompts,
- the failure modes you found, and which ones you fixed, and
- what would break first with 10,000 documents, and how you would fix it.

11.2 How to submit

- Before the deadline, share the GitHub repo. If you'd like to keep it as a private repo, zip the code and share it with us via email.
- You do not need to include the `./data` folder.
- Never commit real API keys.
- We review the last commit made before the deadline. We may also look at your commit history.

12. How we will evaluate

We first run an automated test suite against your service, using our own data and a modified corpus. Then we review your code, tests and documents.

12.1 Must-pass checks

If any of these fails, we stop the review.

1. `docker compose up` starts the service, and `/health` returns 200 within 10 minutes.
2. No document the requester is not allowed to read is ever sent to the LLM.
3. No employee can see leave data they are not allowed to see.
4. No LLM call contains more than 3,000 tokens of corpus text.

5. The service works on the modified corpus, and its answers reflect the changed values.

13. Frequently asked questions

Question	Answer
Can I use LangChain, LlamaIndex or another framework?	Yes.
Do I need a user interface?	No. The API is enough.
Can I change the database schema?	You may add tables and indexes. Do not change or drop existing tables or columns.
Can I convert any documents by hand, for example by typing out the scanned memo?	No. Ingestion must be fully automatic.
Can I store holiday dates in a database table?	Yes, if they are extracted automatically during ingestion.
What counts as “the same rule” in Section 5?	Two documents that state different values for the same thing, such as the number of sick days. Recognising this is part of the task.
Can I reuse code I have written before?	Yes, if you have the right to share it.
What if I run out of time?	Make sure the must-pass checks in Section 12.1 pass first. Then list anything unfinished under known limitations in your README.
Something is unclear. What should I do?	Make a reasonable assumption and record it in your README.